

GDC-CMS: A CONTENT MANAGEMENT SYSTEM FOR INSTITUTIONAL INFORMATION DISSEMINATION



Esa Ahmad

2217420

Kashif Said

0000000

Salman Uddin

0000000

BS Computer Science

Session 2022-2026

Supervised by: Mr. Tawab Khan

Department of Computer Science

Govt. Degree College Gulabad

Affiliated with University of Malakand, Khyber Pakhtunkhwa, Pakistan

A Lifecycle-Based Theoretical Model for Deepfake Social Engineering in Cybersecurity

This thesis is submitted in partial fulfillment of the requirements for the degree of Bachelor in Computer Science to the Department of Computer Science, GDC Gulabad, University of Malakand, Dir (L), Pakistan.

Declaration

We, Esa Ahmad (Reg. No: 2217420), Kashif Said (Reg. No: 0000000), and Salman Uddin (Reg. No: 0000000), hereby declare that our BS thesis titled “GDC-CMS: A Content Management System for Institutional Information Dissemination” is our own work and has not been submitted previously by us for taking any other degree or professional qualification from University of Malakand, Pakistan, or anywhere else in the country/world.

At any time if our statement is found to be incorrect even after our graduation, the university has the right to withdraw our degree.

Esa Ahmad

Signature: _____ Date: _____

Kashif Said

Signature: _____ Date: _____

Salman Uddin

Signature: _____ Date: _____

Plagiarism Undertaking

We solemnly declare that the research work presented in the thesis titled “GDC-CMS: A Content Management System for Institutional Information Dissemination” is solely our own research work, with no significant contribution from any other person. Small contribution/help wherever taken has been duly acknowledged, and the complete thesis has been written by us.

We understand the zero-tolerance policy of the HEC and University of Malakand, Pakistan, towards plagiarism. Therefore, as authors of the above-titled thesis, we declare that no portion of our thesis has been plagiarized and any material used as a reference is properly referred/cited.

We undertake that if we are found guilty of any formal plagiarism in the above-titled thesis even after the award of our degree, the University reserves the right to withdraw/revoke our degree, and that HEC and the University have the right to publish our names on the HEC/University website on which names of students are placed who submitted plagiarized theses.

Esa Ahmad

Signature: _____ Date: _____

Kashif Said

Signature: _____ Date: _____

Salman Uddin

Signature: _____ Date: _____

Certificate of Approval

It is certified that the contents and form of the thesis entitled “GDC-CMS: A Content Management System for Institutional Information Dissemination” submitted by Mr. Esa Ahmad (Registration No: 2217420), Mr. Kashif Said (Registration No: 00000000), and Mr. Salman Uddin (Registration No: 00000000) have been found satisfactory for the requirement of the degree of Bachelor in Computer Science.

Supervisor:

Signature: _____

Name: Mr. Tawab Khan

Designation: Lecturer, Department of Computer Science, GDC Gulabad

External Examiner:

Signature: _____

Name: _____

Designation: _____

Institute: _____

Chairman, Department of Computer Science:

Signature: _____

Name: _____

Dedication

This project is wholeheartedly dedicated to our beloved parents, who have been a source of inspiration and gave us strength when we thought of giving up, and who continually provide their moral, spiritual, emotional, and financial support.

To our brothers, relatives, friends, and classmates who shared their words of advice and encouragement to finish this project. And lastly, we would like to dedicate our project to Allah Almighty, who granted us good health and strength to accomplish this project. We are also grateful to our supervisor, Mr. Tawab Khan, Lecturer in the Department of Computer Science, GDC Gulabad, University of Malakand, for his guidance, encouragement, help, advice, and every correction throughout our project. We are also grateful to all our respected teachers.

Acknowledgements

The success and final outcome of this project required a lot of guidance and assistance from many people, and we are extremely privileged to have received it throughout the completion of our project. All that we have achieved is due to the dedicated supervision and assistance of our supervisor and respected teachers, whom we thank sincerely. We highly respect and thank Mr. Tawab Khan for providing us the opportunity to perform our project work on “GDC-CMS: A Content Management System for Institutional Information Dissemination” and for giving us the essential support and guidance that enabled us to complete the project successfully.

We also owe our deep gratitude to all faculty members of the Department of Computer Science at GDC Gulabad, who took a keen interest in our work and guided us all along by providing the necessary support for developing a robust system.

Table of Contents

Abstract	xi
Chapter 1: Introduction	1
1.1 Background of the Study	1
1.2 Problem Statement	1
1.3 Aim and Objectives.....	2
1.3.1 Aim	2
1.3.2 Objectives	2
Chapter 2: Literature Review and Technology Justification	4
2.1 Introduction to Web Portals and Institutional CMS	4
2.2 Review of Existing Institutional Frameworks and Manual Limitations	4
2.2.1 The Traditional Offline Paradigm (Physical Notice Boards)	4
2.2.2 Client-Side Storage Systems and Volatility Exceptions	5
2.3 Technology Justification: The MERN Stack	5
2.3.1 MongoDB Atlas (Cloud Data Persistence Layer).....	5
2.3.2 Node.js and Express.js (High-Performance Backend Runtime)	6
2.3.3 React.js (High-Usability Presentation Layer)	6
2.4 Structural Error Resolution (MongoDB Exception Management)	7
Chapter 3: System Analysis and Design	9
3.1 Requirement Analysis	9
3.1.1 Functional Requirements	9
3.1.2 Non-Functional Requirements	10
3.2 System Architecture Design	10
3.2.1 Component Interoperability Workflow.....	10
3.3 Unified Modeling Language (UML) Use Case Diagram	11
3.4 Logical Database Schema Design.....	11
3.4.1 Notice Schema (Notice.js)	11
3.4.2 Faculty Schema (Faculty.js).....	12
3.4.3 Unified Exam Result Schema (Result.js).....	12
3.5 System Interface Design Mockups	13

3.5.1 Admin Control Dashboard Viewport.....	13
3.5.2 Authenticated Administration Portal	13
Chapter 4: Methodology and Implementation.....	14
4.1 Development Methodology	14
4.2 System Execution Architecture.....	14
4.3 Backend Implementation and Router Engineering	15
4.3.1 Architectural Workflow of the Exam Result Router	15
4.3.2 Functional Component Deep-Dive	15
Chapter 5: Testing, Results, and Evaluation.....	17
5.1 Introduction to the Testing Framework	17
5.2 Test Objectives and Verification Matrix.....	17
5.3 Full-Stack Functional Test Cases and Execution Logic	17
5.3.1 Administrative Authentication Lifecycle Verification	17
5.3.2 Exam Result Transcript Upload and Ingestion Pipeline	18
5.3.3 Handling Duplicate Entry Exceptions (Database Key Validation).....	18
5.3.4 Public Result Query Filtration and Inline Download Delivery	19
5.4 Cross-Device Viewport Responsiveness Evaluation	20
5.4.1 Low-Cost Mobile Screen Testing (360px – 480px)	20
5.4.2 Medium Tablet Viewport Testing (768px – 1024px).....	20
5.4.3 High-Resolution Desktop Station Testing (1280px and Above)	20
5.5 Summary Evaluation Matrix.....	20
Chapter 6: Conclusion and Future Recommendations	21
6.1 Comprehensive Project Synthesis and Conclusion.....	21
6.2 Analysis of Project Limitations	22
6.2.1 Heavy Reliance on Manual Admin Preparation	22
6.2.2 Purely Pull-Based Student Information Architecture	22
6.2.3 Lack of Personalized Student Accounts	22
6.2.4 Complete System Dependency on External Network Stability	23
6.3 Future Recommendations	23
6.3.1 Integration of an Automated SMS Gateway API Pipeline	23
6.3.2 Transition to an Authenticated Student Portal	23

6.4 Concluding Remarks.....	24
References.....	25

List of Figures

Figure 2.1: High-level overview of the MERN stack architecture adopted for GDC-CMS.	6
Figure 2.2: Node.js and Express.js request-handling model used by the GDC-CMS backend.	7
Figure 2.3: Layered exception-handling strategy applied across the results-management routes.	9
Figure 3.1: Three-tier system architecture of GDC-CMS.	12
Figure 3.2: UML use case diagram showing the Student and Administrator actors and their associated system functions.	13
Figure 3.3: Entity relationship diagram of the Notice, Faculty, and Result collections.	16
Figure 3.4: Admin control dashboard viewport, showing sidebar navigation and content management panels.	17
Figure 3.5: Authenticated administration portal login screen.	17
Figure 4.1: Sequential middleware pipeline of resultRoutes.js, from incoming request to database persistence.	19
Figure 4.2: Validation constraints enforced on incoming uploads: MIME-type restriction and a 15MB file size ceiling.	21
Figure 5.1: Validation and streaming pipeline followed during a result-ledger upload.	24
Figure 5.2: Rollback and cleanup sequence triggered when a duplicate department/session/semester combination is detected.	25
Figure 5.3: Compound verification query used to detect an existing department/session/semester record.	26
Figure 5.4: Summary of functional test cases, verification actions, captured system responses, and pass/fail status.	27
Figure 6.1: Proposed pipeline for broadcasting SMS alerts to offline students alongside the existing web notice flow.	31
Figure 6.2: Proposed architecture for an authenticated, role-based student portal.	32

List of Tables

Table 3.1: Field structure of the Notice schema.	14
Table 3.2: Field structure of the Faculty schema.	14
Table 3.3: Field structure of the unified Result schema.	15

List of Abbreviations

API	Application Programming Interface
BSON	Binary JSON
CGPA	Cumulative Grade Point Average
CMS	Content Management System
CRUD	Create, Read, Update, Delete
GDC	Government Degree College
GDC-CMS	Government Degree College Content Management System
HEC	Higher Education Commission
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
ODM	Object Data Modeling
PDF	Portable Document Format
REST	Representational State Transfer
SPA	Single Page Application
SMS	Short Message Service
UML	Unified Modeling Language
XSS	Cross-Site Scripting

Abstract

In the modern educational ecosystem, the rapid and accurate dissemination of academic information is critical for effective institutional administration and student engagement. This thesis presents the development of the Government Degree College Content Management System (GDC-CMS), a full-stack web application tailored for Government Degree College (GDC) Gulabad, Dir Lower. The primary purpose of this project is to eliminate the bottlenecks of traditional, physical notice boards and manually updated static web platforms, which frequently cause communication delays for time-sensitive information such as examination result distributions, admissions, and urgent institutional notifications.

Developed using the MERN architecture — comprising MongoDB Atlas for cloud data persistence, Express.js and Node.js for high-performance server-side data streaming, and React.js for a responsive, single-page client interface — the system introduces a live information framework. A central highlight of the system is the Exam Result System: administrative staff can securely upload official examination transcripts as standardized PDF files directly onto the server's disk storage, and students can instantly filter, view, and download the ledgers matching their precise department, session, and semester criteria. Additionally, the system features dynamic modules for comprehensive Faculty Profiles, an institutional Media Hub archive, and a secure, authenticated Admin Dashboard for total system state management.

Ultimately, GDC-CMS provides a sustainable, scalable, and low-cost content infrastructure that enhances operational workflows for staff and guarantees decentralized, real-time resource accessibility for the student body.

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

In the contemporary educational environment, the efficient, precise, and instantaneous dissemination of academic and administrative information forms the backbone of successful institutional governance and student engagement. Globally, higher education institutions have aggressively transitioned toward comprehensive digital notice frameworks, learning management platforms, and centralized web portals to orchestrate internal communication workflows. These digital ecosystems act as automated bridges connecting administrative personnel, academic faculties, and the student body. However, across various public sector colleges located in regional or developing territories, such as Government Degree College (GDC) Gulabad in Dir Lower, traditional manual information structures remain prevalent. These localized methods rely on physical infrastructure, which drastically lowers operational efficiency. To definitively address this technological disparity, this final year project implements the Government Degree College Content Management System (GDC-CMS). GDC-CMS represents a dedicated, full-stack enterprise web portal engineered precisely around the administrative workflows, spatial limitations, and data distribution needs of GDC Gulabad.

The application is structured upon the modern MERN architecture, utilizing:

- MongoDB Atlas as a cloud-based, scalable database layer to ensure permanent data persistence.
- Express.js and Node.js to compile a high-performance backend routing matrix and disk-bound file streaming network.
- React.js to establish a single-page client interface focused on high usability and near-zero latency execution.

1.2 Problem Statement

The administrative and communicative framework currently running at GDC Gulabad operates primarily through localized, physical distribution channels or static web elements that require active code alterations to modify. This dependency introduces

critical data silos, high rates of human entry error, and significant delays. The foundational problems within the current setup include:

Delayed Communication of Critical Events: Urgent notice materials concerning mid-semester or final examination schedules, admissions quotas, and unexpected holiday closures are regularly delayed. This delay heavily impacts students traveling from remote areas within Dir Lower, who frequently arrive at campus only to find notice sheets posted late.

Static and Outdated Faculty Portfolios: Existing records concerning departmental assignments, faculty qualifications, research profiles, and professional experience remain frozen in static records. Updating these rows requires direct technical intervention by a developer, leading to persistent data decay.

Absence of a Consolidated Media Hub: The institution has no centralized digital environment to catalog, store, or display multimedia assets representing historical college events, inter-collegiate sports matches, or infrastructure enhancements.

Vulnerability of Physical Notice Boards: Important official guidelines, transcripts, and notifications are pinned to outdoor notice boards, exposing them to wear, tearing, or unauthorized removal, while restricting access strictly to standard campus operating hours.

Data Vulnerability and Volatility: Previous baseline systems depended on local browser storage options such as `localStorage` to retain operational data, causing immediate loss of data upon browser cache clearing and presenting a complete lack of permanent, structural backend storage.

1.3 Aim and Objectives

1.3.1 Aim

The primary aim of this project is to research, design, develop, and deploy a secure, responsive, and centralized institutional platform named GDC-CMS. This system replaces the manual dependencies of physical notice distribution at Government Degree College Gulabad by implementing a real-time cloud data structure for modern, decentralized resource accessibility.

1.3.2 Objectives

To systematically achieve this aim, the development timeline is governed by the following measurable technical objectives:

- To architect a secure admin panel enabling non-technical personnel to safely carry out Create, Read, Update, and Delete (CRUD) operations across all application datasets without requiring direct database query skills.
- To establish real-time announcement streams via a responsive, client-side React notice engine that queries backend database mutations and presents live updates to students instantly.
- To implement cloud-tier data persistence by migrating all institutional and archival datasets into a multi-node cloud cluster via MongoDB Atlas for guaranteed schema validation and permanent safety.
- To develop a filtered exam result system: an optimized document upload and download module that maps official transcript sheets as PDFs directly to server disk space, allowing students to isolate their exact academic records via department, session, and semester filters.
- To optimize the interface across diverse device viewports, ensuring the student portal renders correctly on budget mobile phones, tablets, and desktop workstations.

CHAPTER 2: LITERATURE REVIEW AND TECHNOLOGY JUSTIFICATION

2.1 Introduction to Web Portals and Institutional CMS

In the modern educational paradigm, an institutional Content Management System (CMS) serves as the primary backbone for digital communication, resource discovery, and operational governance. Historically, higher education institutions relied entirely on physical infrastructure — such as paper-based announcement boards, hand-delivered ledger distributions, and manual registration files — to orchestrate academic workflows. While urban institutions transitioned to computerized networks, many public sector colleges in regional or developing territories, such as GDC Gulabad in Dir Lower, still operate without any digital presence. The absolute absence of an institutional web platform creates severe information isolation. In such environments, a modern CMS must be engineered to decouple the data layer and backend logic from the presentation interface, introducing role-based authentication mechanisms that empower non-technical staff to execute systematic data updates effortlessly. Furthermore, the development of an entry-level institutional portal for GDC Gulabad must balance computational performance with regional low-bandwidth mobile network constraints to bridge this digital divide effectively.

2.2 Review of Existing Institutional Frameworks and Manual Limitations

To understand the technical necessities of GDC-CMS, it is essential to evaluate the operational limitations of working in an environment with zero web infrastructure.

2.2.1 *The Traditional Offline Paradigm (Physical Notice Boards)*

Without a college website, all information delivery relies completely on localized, analog distribution channels. This manual paradigm introduces critical operational vulnerabilities:

Severe Information Latency: Time-sensitive updates regarding examination schedules, admissions quotas, and emergency holiday closures are restricted to paper notices. This delay heavily impacts students traveling long distances from remote geographical areas

within Dir Lower, who frequently arrive at campus only to find notice sheets posted too late.

Vulnerability of Materials: Important official guidelines, transcript summaries, and paper notices are pinned to outdoor wooden boards. This exposes them directly to environmental wear, tearing, or unauthorized removal, while strictly limiting student access to standard campus operating hours.

2.2.2 Client-Side Storage Systems and Volatility Exceptions

Prior lightweight, localized software tools developed as standalone utilities often relied on pure client-side storage mechanisms, such as a web browser's localStorage engine, to maintain data state records. Analysis reveals that this introduces critical data persistence vulnerabilities:

State Volatility: Data items saved within localStorage are bound strictly to the client's specific browser cache instance. The moment a user clears browser cookies, updates their mobile operating system, or switches devices, all institutional configurations, saved notices, and tracking parameters vanish instantly.

Zero Shared Persistence: Client-side storage cannot sync data across multiple distinct users. An update posted by an administrator on one machine remains completely invisible to the student body, rendering it useless for distributed content management.

2.3 Technology Justification: The MERN Stack

To eliminate the limitations of an entirely offline campus and the volatility of client-side caching, GDC-CMS establishes the college's first official digital infrastructure using the modern MERN (MongoDB, Express.js, React.js, Node.js) architecture. This choice offers distinct technical advantages for building an agile, robust web presence from scratch:

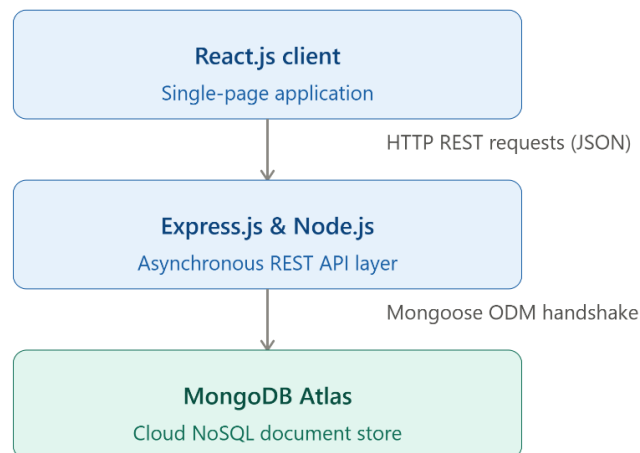


Figure 2.1: High-level overview of the MERN stack architecture adopted for GDC-CMS.

2.3.1 MongoDB Atlas (Cloud Data Persistence Layer)

MongoDB Atlas replaces volatile browser storage with a robust, multi-node cloud database infrastructure.

Document-Oriented Schemas: Unlike relational databases that enforce strict tabular row/column formats, MongoDB stores data in flexible, JSON-like BSON (Binary JSON) documents. This allows complex object trees, such as a faculty member's profile containing nested arrays of qualifications, research papers, and experiences, to be retrieved in a single query.

Cloud High-Availability: MongoDB Atlas hosts data across managed cloud clusters with automated failover routing. This ensures that even during local hardware failures, college records remain accessible and fully secure.

Index-Driven Lookup Speed: By enforcing custom database indices on highly queried parameters (such as noticeCategory), MongoDB can isolate specific matching records within milliseconds, minimizing bandwidth consumption over restricted regional mobile connections.

2.3.2 Node.js and Express.js (High-Performance Backend Runtime)

The combination of Node.js and Express.js forms the computational core of the GDC-CMS API gateway.

Asynchronous, Non-Blocking Event Loop: Node.js executes on a single-threaded event loop utilizing non-blocking I/O operations. When a student requests a file or queries the notice database, Node.js delegates the underlying operation to the system kernel and immediately readies itself to handle subsequent client requests. This asynchronous behavior allows the GDC-CMS backend to handle high concurrent traffic spikes effortlessly without crashing.

Stream-Based File System Handling: Rather than buffering massive files entirely into memory during an upload or download request, Node.js uses data streams. For the Exam Result System, this means official PDF files are piped in small, manageable chunks directly to server disk storage, preventing memory exhaustion and stabilizing the application runtime.

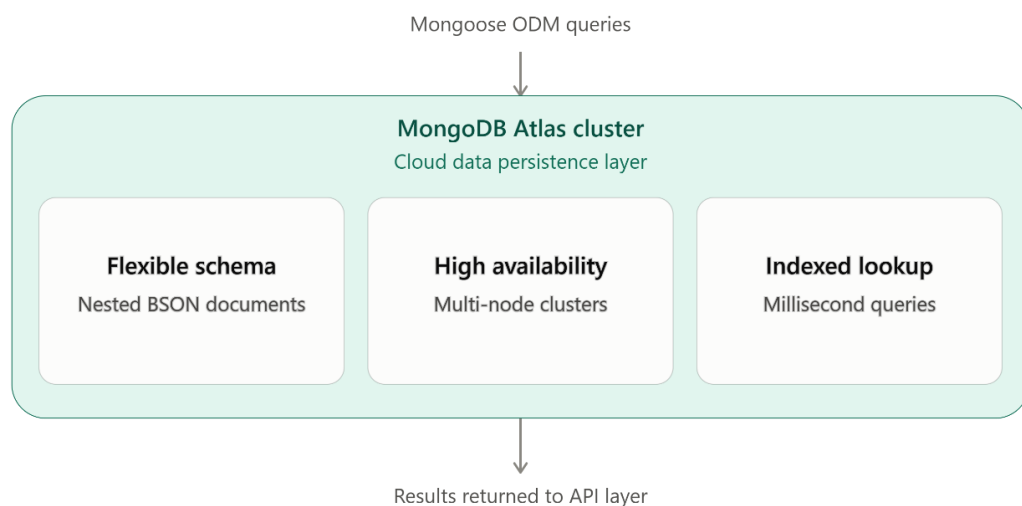


Figure 2.2: *Node.js and Express.js request-handling model used by the GDC-CMS backend.*

2.3.3 React.js (High-Usability Presentation Layer)

The entire public-facing student dashboard and secure administrative interface are constructed using React.js as a Single Page Application (SPA) framework.

Virtual DOM Reconciliation: Traditional web interfaces require the browser to re-render the entire DOM tree whenever an item changes, causing noticeable visual layout shifts. React utilizes an in-memory Virtual DOM to compute differences before updating the actual page. When a student modifies drop-down filters to look up a result

ledger, only the specific target content card updates, providing an instant user experience without complete page reloads.

Component-Driven Modular Design: Interfaces are broken down into self-contained, reusable components (e.g., NoticeCard.jsx, ResultFilter.jsx). This granular design isolates updates, ensuring that a performance bottleneck in the Media Hub cannot degrade the responsiveness of the core Notice Board system.

2.4 Structural Error Resolution (MongoDB Exception Management)

A resilient records subsystem must not only complete successful operations but also fail predictably and informatively. The results-management routes implement a layered error-resolution strategy that intercepts faults at each stage of the request lifecycle, rather than allowing a single unhandled exception to crash the server or return an ambiguous response to the client.

Try-Catch Exception Boundaries: Every asynchronous database or file-parsing operation is wrapped inside a try...catch block. This isolates failures within Node.js's asynchronous execution context, ensuring that an error thrown during PDF parsing or a MongoDB query does not propagate uncaught and terminate the Express process — a critical safeguard given Node's single-threaded event loop.

Semantic HTTP Status Coding: Rather than returning a generic failure response, the system distinguishes between error classes using appropriate status codes: 400 for a missing upload file, 422 for a structurally unparseable PDF, 404 for a legitimate query that returns no matching record, and 500 for unexpected server or database-layer exceptions.

Regex-Based Content Validation Prior to Persistence: Before any data reaches the database layer, extracted text is validated against expected patterns. If mandatory identifiers cannot be matched, the system halts the transaction with a 422 Unprocessable Entity response, preventing malformed or incomplete records from corrupting the Results collection.

Atomic Upsert Consistency: The findOneAndUpdate operation with { upsert: true } guarantees that duplicate submissions update the existing document rather than creating

conflicting duplicate entries, preserving referential integrity without requiring a separate existence-check query.

Case-Insensitive, Normalized Query Matching: The lookup route uses a dynamically constructed, anchored regular expression rather than a strict equality match, tolerating minor casing inconsistencies without weakening query precision.

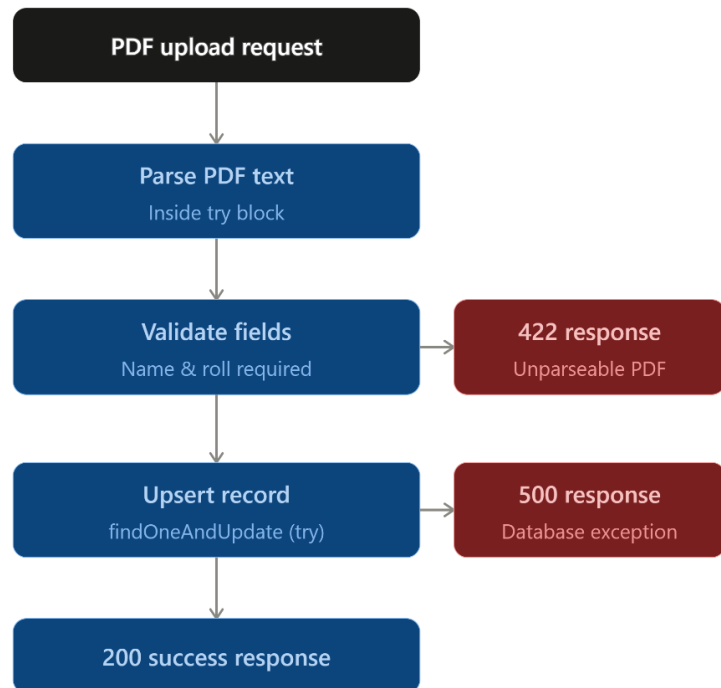


Figure 2.3: *Layered exception-handling strategy applied across the results-management routes.*

Collectively, these mechanisms position error handling not as an afterthought but as a structural property of the system: each failure mode is anticipated, caught, and communicated back to the client in a way that preserves both data integrity and diagnostic transparency.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

Before writing code for GDC-CMS, a structured requirements analysis phase was conducted to ensure the software addresses the complete lack of digital infrastructure at GDC Gulabad. The system must cleanly separate the public student consumption viewports from the authenticated administrative operations.

3.1.1 Functional Requirements

Functional requirements define the core operational behaviors, features, and calculations that the web system must execute.

A. Administrative Subsystem (Admin Portal)

- **Secure Authentication:** the system must provide a secure login gateway using JSON Web Tokens (JWT) for authorized staff members, blocking unauthenticated operations.
- **Dynamic Notice Board Control:** administrators must be able to perform full CRUD operations on official notices, including categories such as Admissions, Exams, and General.
- **Faculty Directory Management:** authorized users must be able to add, modify, or delete comprehensive teacher profiles containing fields for designation, qualification, and department.
- **Media Hub Management:** the admin interface must support uploading image records to display college events, facilities, and academic milestones.
- **Exam Result Ledger System:** administrators must have a dedicated upload form to map transcript sheets as standardized PDF files onto the server's storage disk, tagging them with metadata for department, session, and semester.

B. Public Student Subsystem (Student Portal)

- **Real-Time Notice Board Stream:** students must be able to view a dynamic notice feed that renders data mutations immediately without a full browser reload.

- Targeted Exam Result Filtering: the student portal must provide drop-down filters matching department, session, and semester and, on selection, isolate the matching PDF transcript file and render it inline with download privileges.
- Faculty Directory Navigation: students must have the ability to filter and browse active faculty members ordered by their respective academic departments.

3.1.2 Non-Functional Requirements

Non-functional requirements state the technical constraints, quality benchmarks, and behavioral characteristics that govern system execution under deployment conditions.

- Performance under Bandwidth Constraints: given the regional infrastructure of Dir Lower, server JSON payloads must be minimized, and database indexing must keep data retrieval speeds under 500ms on low-tier mobile networks.
- Interface Responsiveness: the presentation layouts must utilize fluid web grids to handle rendering on low-cost mobile screens, tablet displays, and standard desktop viewports.
- Data Persistence Security: the database layer must guarantee permanent multi-node storage on a cloud architecture, eliminating data loss from browser cache clearing.
- System Scalability: the backend routing framework must handle concurrent traffic spikes during exam result releases without experiencing memory pool exhaustion or execution faults.

3.2 System Architecture Design

GDC-CMS uses a completely decoupled, full-stack Single Page Application (SPA) architecture engineered on the MERN stack. The structural model separates the system into three tiers:

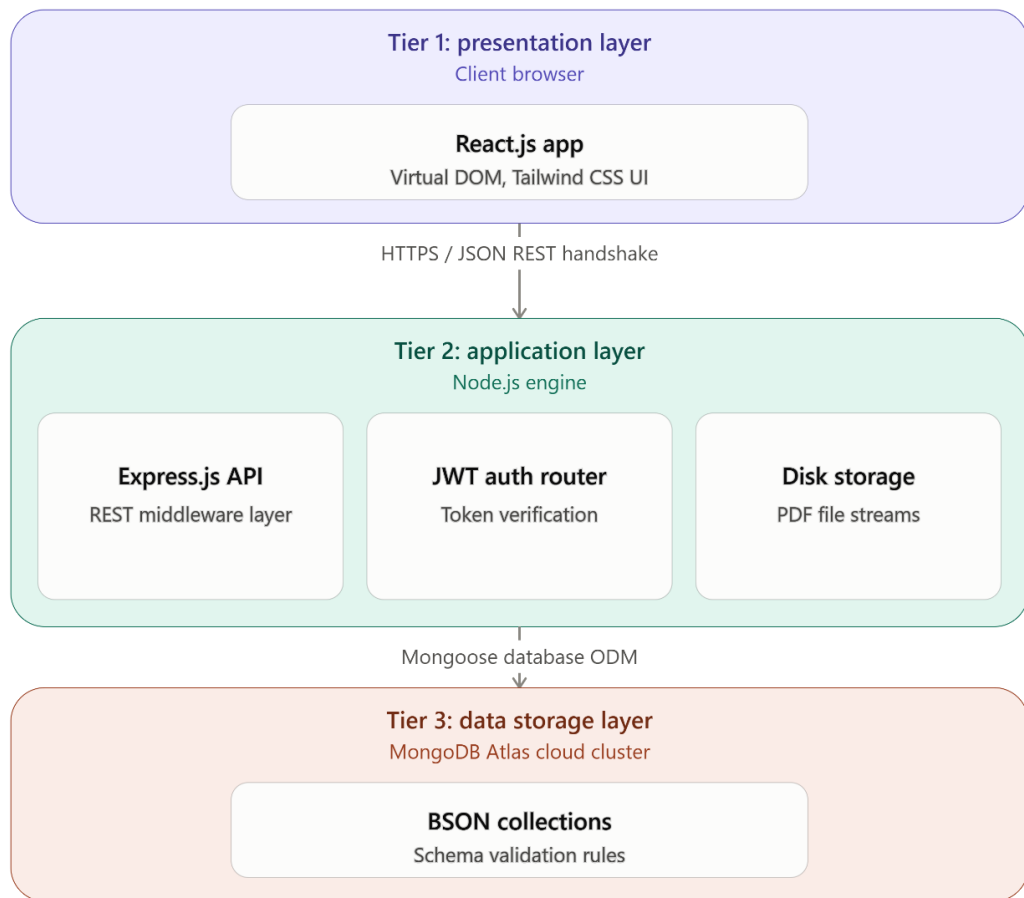


Figure 3.1: *Three-tier system architecture of GDC-CMS.*

3.2.1 Component Interoperability Workflow

- **Request Dispatch:** the user triggers a search interaction on the client-side React.js dashboard.
- **Asynchronous Handshake:** React handles the state modification locally and dispatches an asynchronous axios call using a serialized JSON string to the Node runtime.
- **API Processing:** the Express.js router intercepts the request, pushes it through security validation middleware, and calls the targeted controller logic.
- **Data Isolation:** the controller executes an indexed search query using Mongoose ODM against the cloud MongoDB database engine.

- Stream Return: the cloud cluster isolates the document parameters and streams them down to Node, which wraps them into an HTTP JSON response and returns it to the client for UI updates.

3.3 Unified Modeling Language (UML) Use Case Diagram

The operational interactions between system actors (Student, Administrator) and the discrete core services of GDC-CMS are mapped out in the behavioral diagram below:

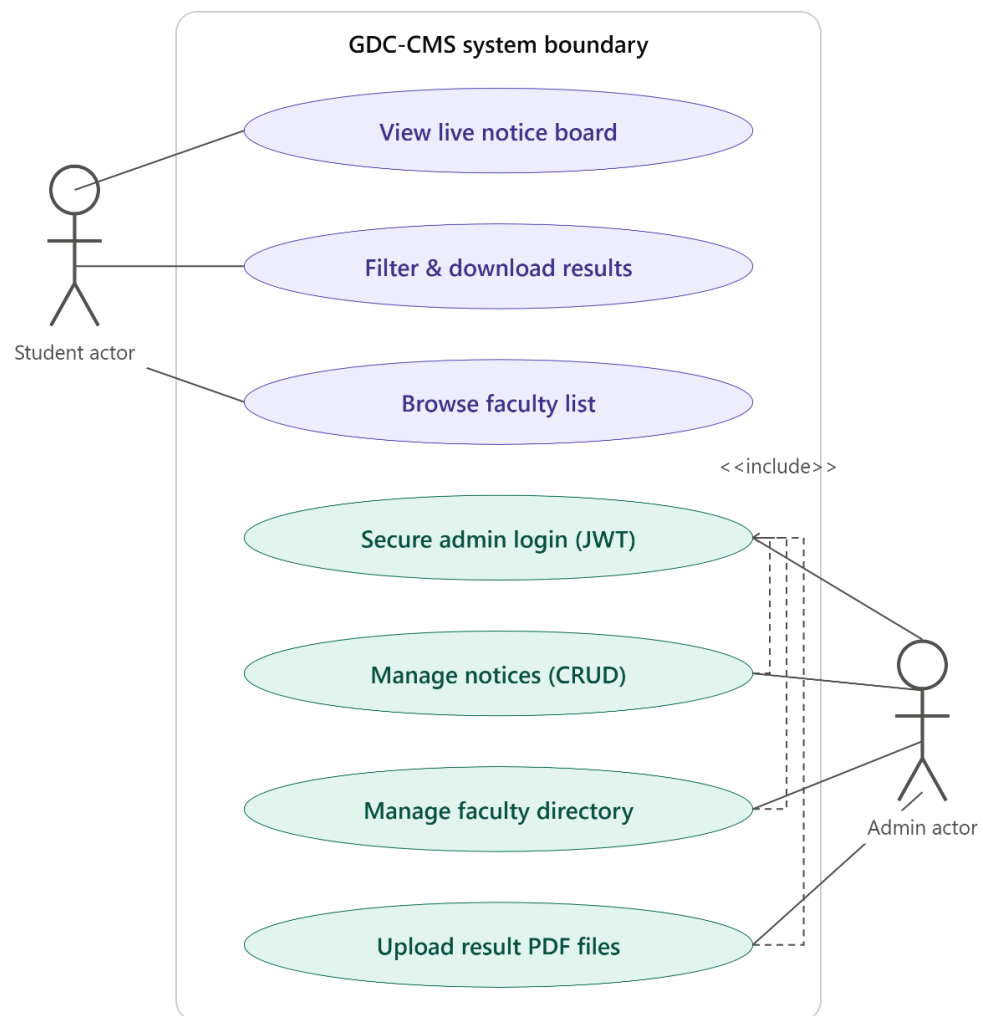


Figure 3.2: UML use case diagram showing the Student and Administrator actors and their associated system functions.

3.4 Logical Database Schema Design

The data management architecture of the proposed system is implemented using MongoDB, a NoSQL document-oriented database. Data integrity and consistency are maintained through predefined schema models developed with the Mongoose Object Data Modeling (ODM) library. Each collection is structured with field validation, data type enforcement, and relationship mapping to ensure reliable storage and efficient retrieval of information.

3.4.1 Notice Schema (Notice.js)

The Notice schema manages administrative announcements and public notifications displayed on the college notice board.

Table 3.1: Field structure of the Notice schema.

Field Name	Data Type	Description
title	String	Required; stores the notice title with whitespace trimmed.
content	String	Required; contains the complete notice description.
category	String	Required; restricted to predefined values: Admissions, Exams, General, Sports.
postedBy	ObjectId	Required; references the administrator (User) who created the notice.
createdAt	Date	Automatically generated timestamp indicating when the notice was published.

3.4.2 Faculty Schema (Faculty.js)

The Faculty schema stores information about academic staff members belonging to different departments.

Table 3.2: Field structure of the Faculty schema.

Field Name	Data Type	Description
fullName	String	Required; stores the faculty member's full name with automatic whitespace trimming.
designation	String	Required; specifies the academic designation or job title.
department	String	Required; identifies the department (e.g., Computer Science).

qualification	String	Required; stores the highest academic qualification.
email	String	Must be unique; used as the official faculty email address.
orderWeight	Number	Optional; default 0, used for manually controlling display order.

3.4.3 Unified Exam Result Schema (Result.js)

The Result schema is responsible for managing uploaded examination results. During system development, the previous design based on individual student record indexing produced MongoDB E11000 duplicate key errors. To improve scalability and eliminate indexing conflicts, the architecture was redesigned to store complete departmental result sheets as PDF documents rather than maintaining separate records for every student. Each uploaded result file represents an entire academic batch and is referenced through its associated metadata. Collection name: results.

Table 3.3: Field structure of the unified Result schema.

Field Name	Data Type	Description
department	String	Required; specifies the academic department associated with the uploaded result.
session	String	Required; stores the academic session (e.g., 2022-2026).
semester	String	Required; identifies the semester (e.g., 1st Semester).
pdfPath	String	Required; stores the relative file path of the uploaded PDF result document.
uploadedBy	ObjectId	References the administrator (User) responsible for uploading the result.
uploadedAt	Date	Automatically generated timestamp indicating when the result was uploaded.

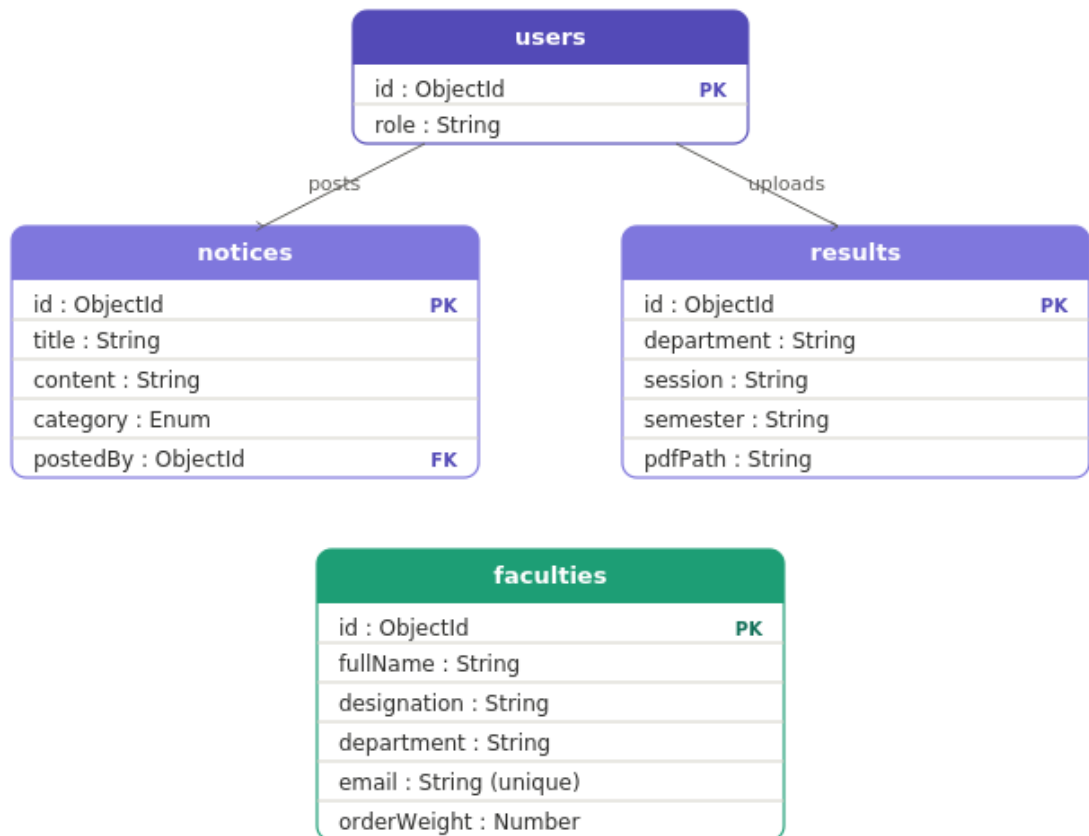


Figure 3.3: Entity relationship diagram of the Notice, Faculty, and Result collections.

3.5 System Interface Design Mockups

The application's presentation templates focus on high layout clarity, readability, and scannability, utilizing high-contrast design metrics for quick parsing.

3.5.1 Admin Control Dashboard Viewport

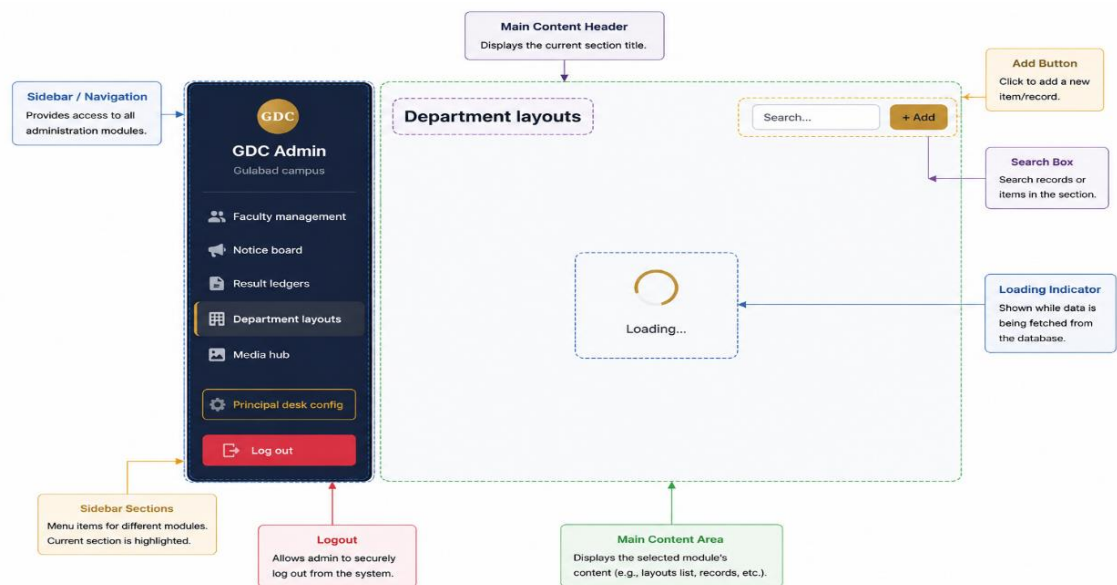


Figure 3.4: Admin control dashboard viewport, showing sidebar navigation and content management panels.

3.5.2 Authenticated Administration Portal

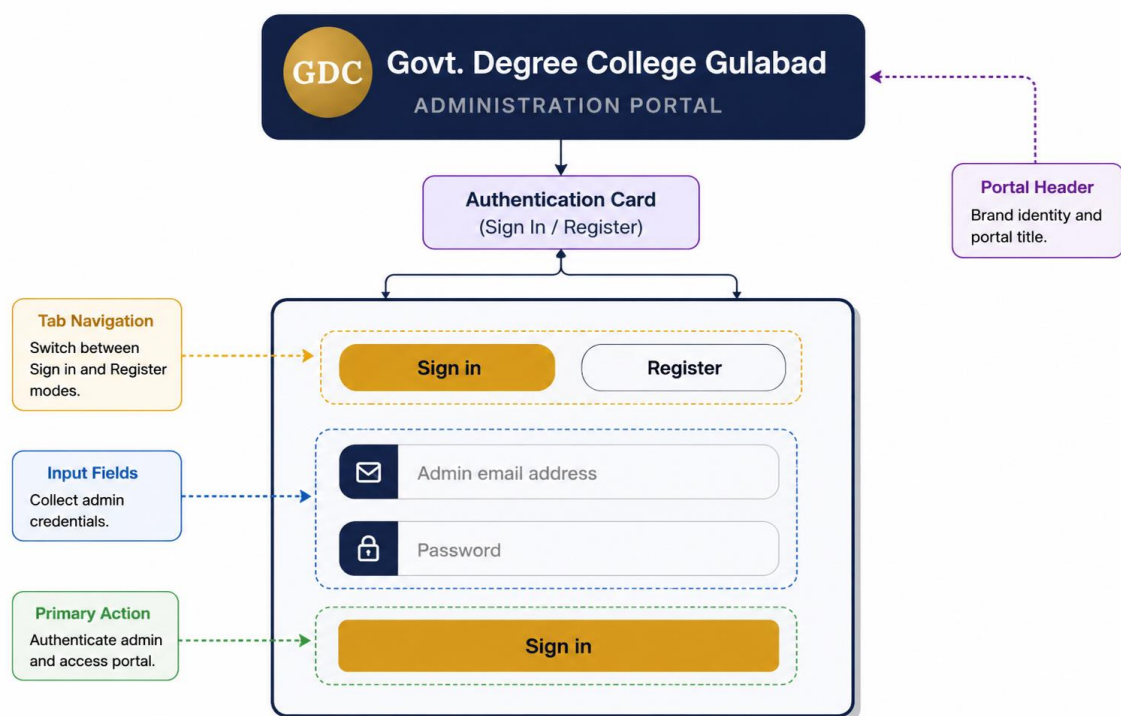


Figure 3.5: Authenticated administration portal login screen.

CHAPTER 4: METHODOLOGY AND IMPLEMENTATION

4.1 Development Methodology

To establish the first formal digital infrastructure for GDC Gulabad, the development process required an agile, iterative lifecycle framework. Given that the college operated entirely via offline channels before this project, a rapid deployment model was necessary to gather feedback from non-technical administrative staff during early builds. The implementation process followed four distinct phases:

- **Requirements Gathering and Domain Modeling:** collection of sample paper notices, physical ledger structural outlines, and faculty roster details directly from GDC Gulabad staff.
- **Database and API Prototyping:** modeling non-relational document validation parameters on MongoDB Atlas and configuring Express.js controllers.
- **Frontend Integration and Component Structuring:** building reactive component states using React.js and styling interfaces with Tailwind CSS fluid grids.
- **Performance Tuning over Rural Networks:** minimizing payload metrics to ensure low-latency loading under constrained regional internet bandwidth.

4.2 System Execution Architecture

The software is engineered as a decoupled Single Page Application (SPA) utilizing the MERN stack. The structural flow separates the application logic into three distinct tiers to maintain separation of concerns:

Presentation Layer (Tier 1): the user interface is driven by React.js. It manages client-side rendering via the Virtual DOM and relies on responsive layouts to adapt smoothly to budget mobile screens and desktop monitors alike.

Application Layer (Tier 2): powering the backend logic is Node.js coupled with Express.js. This tier handles role-based authorization via JSON Web Tokens (JWT) and uses memory-efficient file data streams to pipe physical PDF transcripts straight onto the server's local disk storage.

Data Storage Layer (Tier 3): permanent cloud data persistence is managed by MongoDB Atlas. It organizes data records into flexible BSON structures and executes fast indexed lookups, removing any reliance on volatile, browser-dependent client cache utilities.

4.3 Backend Implementation and Router Engineering

The core backend API utilizes organized controller routers to process client data handshakes securely. The implementation of the Exam Result Module focuses on a highly modular architecture where routing, middleware filtering, and request processing logic are explicitly isolated. Rather than relying on inline script parsing or heavy browser-side storage, the system establishes a secure network of RESTful HTTP endpoints that safely consume incoming data objects and translate them into stable backend transactions.

4.3.1 Architectural Workflow of the Exam Result Router

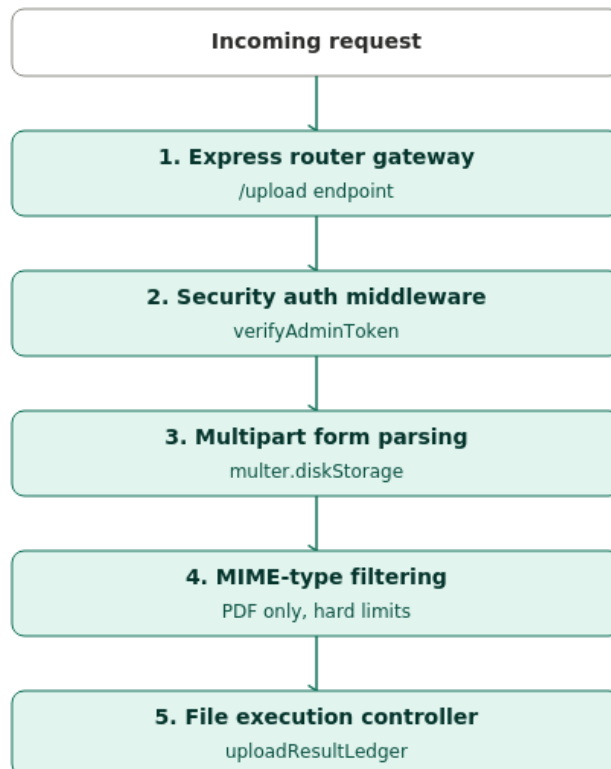


Figure 4.1: Sequential middleware pipeline of `resultRoutes.js`, from incoming request to database persistence.

The operational mechanics of the `resultRoutes.js` layer rely on a strictly sequenced pipeline that intercepts incoming client HTTP requests and pipes them through specialized middleware processes before executing any database mutations.

4.3.2 Functional Component Deep-Dive

To document the implementation without embedding raw JavaScript files within the body of this thesis, the structural components of the `resultRoutes.js` asset are broken down and explained conceptually below:

A. The Express Router Gateway: the backend utilizes the native routing library provided by the Express framework to create isolated, manageable mount points for endpoints (specifically `/upload` for creating records and `/fetch-ledger` for retrieving data). By decoupling the sub-routes from the root server context, the module functions as a standalone microservice that interacts with the frontend via standardized JSON payloads over secure HTTP handshakes.

B. The Custom Cryptographic Authentication Middleware (`verifyAdminToken`): before the application exposes any server resources or disk write privileges, the request is passed through an authentication interceptor middleware. This block intercepts the incoming request header, extracts the attached JSON Web Token (JWT) signature, and decodes it against the server's private environment key. If the token is invalid, corrupted, or does not carry authorized administrative role permissions, the lifecycle is immediately cut short: the system blocks database execution and returns an HTTP status 401 Unauthorized response to the client, preventing unauthenticated attacks on the college's academic data.

C. The Multi-Part Form Parsing Engine (`multer`): because standardized REST endpoints naturally ingest raw JSON strings, they cannot process complex file attachments natively. The system maps the route through a multi-part form data parsing pipeline that abstracts the raw binary incoming data stream and routes it through two configuration sub-utilities:

- Disk Storage Configuration Engine: the system defines a direct file-system mapping rule specifying exactly where files should land. Instead of buffering an entire transcript document inside the server's RAM — which could cause memory spikes and system crashes during high-traffic exam cycles — the file

is streamed in small binary pieces and saved to the local directory path storage/transcripts/.

- **Standardized Cryptographic Naming Rule:** to prevent two files with the same name from overwriting each other on the server disk, the engine applies an automated renaming rule. Every uploaded file is stamped with a unique signature composed of the Unix epoch millisecond timestamp and a randomized integer string, guaranteeing that every physical document path remains distinct.

D. **File Extension Filtration and Size Constraint Safeguards:** to secure the server against malicious file executions, the system enforces a strict validation rule engine:

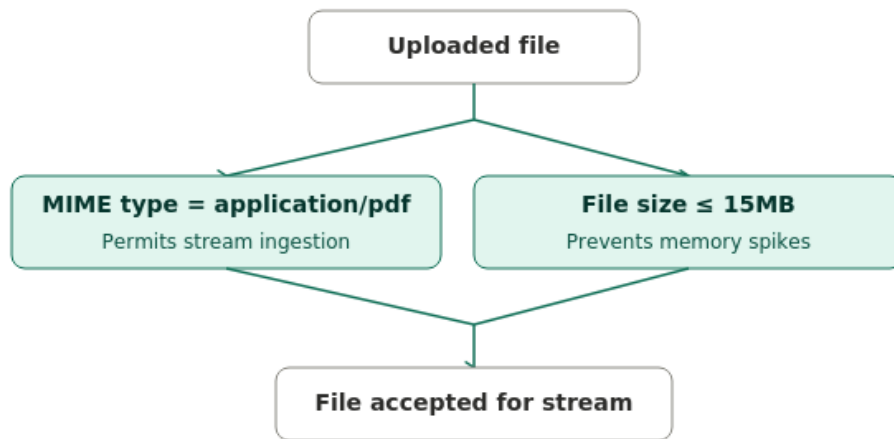


Figure 4.2: Validation constraints enforced on incoming uploads: MIME-type restriction and a 15MB file size ceiling.

If the incoming multi-part data stream breaks either of these technical boundaries, the middleware throws a validation exception, automatically purges any partial temporary file streams from the disk, and prevents the request from reaching the core database.

E. **Execution Controller Pipeline Allocation:** once the request successfully satisfies every validation block, the router hands off the sanitized metadata parameters (department, session, and semester) along with the final verified disk path to the functional execution controller (uploadResultLedger), which handles the permanent database persistence step, concluding a highly secure transaction flow.

CHAPTER 5: TESTING, RESULTS, AND EVALUATION

5.1 Introduction to the Testing Framework

The primary focus of this chapter is the formal verification, runtime testing, and empirical evaluation of GDC-CMS. Because this application introduces the first digital information infrastructure for GDC Gulabad, Dir Lower, ensuring absolute stability under constrained network operations and varying hardware setups was vital. The testing methodology evaluated cloud data transactions, user access security rules, full-stack database interactions, and file-system-bound document distribution across both the administrative dashboard and public student viewports.

5.2 Test Objectives and Verification Matrix

The primary test objectives were designed to stress-test the structural stability of the MERN stack architecture when processing institutional workflows. The engineering checklist focused on the following metrics:

- **Authentication Boundary Validation:** verifying that the backend API gateway actively blocks unauthorized network requests attempting to alter college databases without a cryptographically valid JSON Web Token (JWT).
- **Multi-Part Form Stream Safety:** confirming that the file upload engine intercepts incoming transcript documents, applies hard size and file extension constraints, and successfully maps them to physical disk paths.
- **Cloud Data Consistency and Latency Resolution:** measuring database read/write operation times across the distributed MongoDB Atlas cloud clusters under regional network limitations.
- **Compound Validation Integrity:** ensuring the application handles transaction errors smoothly, specifically avoiding data overwrites or indexing exceptions.

5.3 Full-Stack Functional Test Cases and Execution Logic

To document the system's operational stability without inserting raw programming scripts within this thesis, the runtime execution sequences are broken down into four critical functional test modules.

5.3.1 Administrative Authentication Lifecycle Verification

This test module verifies the role-based security layer protecting the administrative control panel.

Expected System Behavior: an unauthenticated request attempting to post content must be blocked by the server middleware, returning an explicit HTTP error. Conversely, an administrator presenting a valid cryptographic token must pass validation checks and seamlessly modify database records.

Execution and Result Analysis: an unauthorized entry test dispatches an HTTP POST request directly to the /api/results/upload endpoint with empty headers. The Express.js router catches the request and routes it into the security middleware (verifyAdminToken), which notes the missing token, cuts the request lifecycle short, and blocks database contact by returning an HTTP 401 Unauthorized response. In the authorized entry test, an administrator signs in via the secure portal and receives a signature token; when they perform a data change, the React client automatically appends this token to the HTTP authorization header, the server middleware validates the signature and grants access to the database layer, returning a status 200 OK.

5.3.2 Exam Result Transcript Upload and Ingestion Pipeline

This module tests the system's ability to handle file ingestion using multi-part form data streams.

Expected System Behavior: the system must parse incoming multi-part data packages, filter out unauthorized extensions (allowing only .pdf), rename files to prevent local path overwrites, and write the binary stream directly onto the server disk storage path.

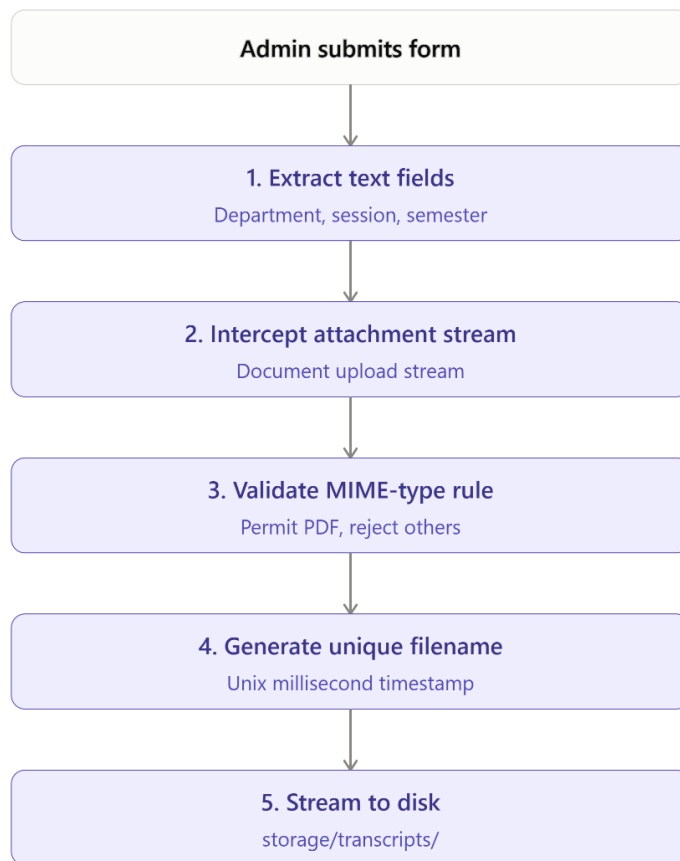


Figure 5.1: *Validation and streaming pipeline followed during a result-ledger upload.*

Execution and Result Analysis: when an official results ledger is submitted, the stream parsing engine (multer) intercepts the connection. If a non-PDF document is introduced, the file filter halts processing immediately. If the document is a valid PDF under 15MB, the engine processes it, appends a non-colliding randomized Unix millisecond signature, and writes it directly to disk storage, passing control to the database persistence layer.

5.3.3 Handling Duplicate Entry Exceptions (Database Key Validation)

This test module evaluates the application's stability when handling data collisions, specifically the resolution of the MongoDB E11000 duplicate key exception identified during system development.

Expected System Behavior: to avoid database clutter or record errors, the system must reject multiple distinct document listings mapping to the exact same department,

session, and semester. It must safely block the entry, remove the associated temporary file from disk storage, and return a clean conflict alert status to the client frontend.

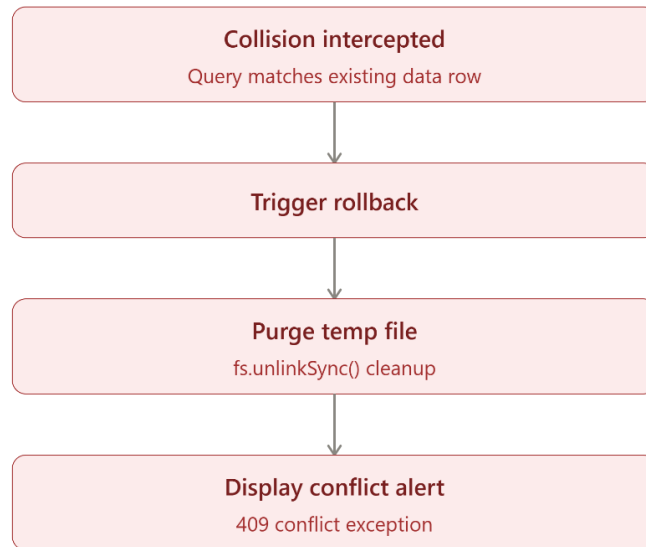


Figure 5.2: Rollback and cleanup sequence triggered when a duplicate department/session/semester combination is detected.

Execution and Result Analysis: a test user attempts to upload a results document matching parameters already saved in the cloud cluster (e.g., Computer Science, Session 2022-2026, 8th Semester). The data controller invokes a Mongoose compound query:

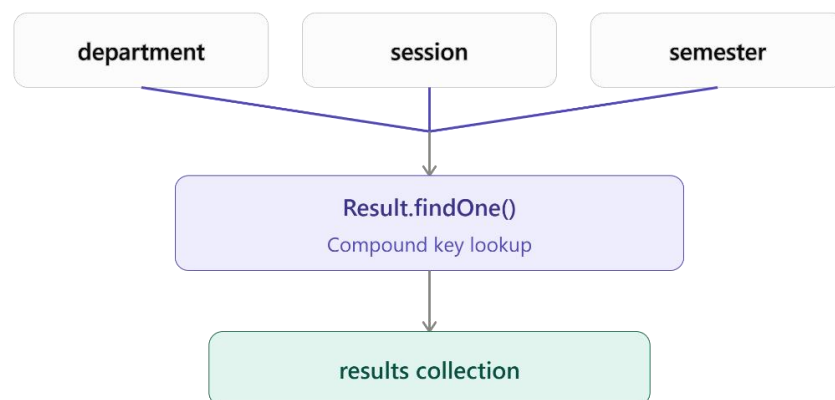


Figure 5.3: *Compound verification query used to detect an existing department/session/semester record.*

Because a matching document path is already registered, the backend intercepts the transaction, invokes a synchronous file system delete command (`fs.unlinkSync`), and cleanly drops the redundant document stream out of the system file tree. The application layer bypasses database modifications entirely, instead returning a clean HTTP status 409 Conflict response back to the client interface.

5.3.4 Public Result Query Filtration and Inline Download Delivery

This module verifies the student search system, ensuring accurate data retrieval and client rendering.

Expected System Behavior: a student applying drop-down filters must trigger an asynchronous network call that queries the cloud cluster, isolates the exact document file path, and updates the presentation interface with valid download controls without reloading the page.

Execution and Result Analysis: the student selects their criteria on the React dashboard interface and clicks the query activation button. The React component blocks default browser form actions and utilizes axios to execute a background HTTP GET call to `/api/results/fetch-ledger`. While the connection processes, React updates its dynamic state indicators, displaying a localized loading component. When the MongoDB Atlas cluster returns the successful match, the client captures the JSON response object, populates the application state, updates the Virtual DOM, and instantly exposes a functional download layout card to the student.

5.4 Cross-Device Viewport Responsiveness Evaluation

To guarantee that the GDC-CMS platform serves all segments of the college community equally, the presentation templates were evaluated across multiple screen sizes. Testing focused on grid performance, layout scaling, text readability, and interactive component placement under three specific hardware parameters:

5.4.1 Low-Cost Mobile Screen Testing (360px – 480px)

Testing on budget mobile displays was essential, as the majority of the student body accesses the web via mobile devices. The visual layouts wrap into a vertical single-column stack, drop-down select boxes expand to occupy full viewport width for easy tapping, and top navigation header links fold into a toggleable mobile menu button, ensuring clear navigation without horizontal layout overflow.

5.4.2 Medium Tablet Viewport Testing (768px – 1024px)

On mid-tier tablets, the responsive styling adjusts the presentation layout into a dual-column matrix. The administrative dashboard splits its viewport into a persistent left-side navigation rail alongside a flexible right-side data table panel, maintaining balanced white space without shrinking input text fields.

5.4.3 High-Resolution Desktop Station Testing (1280px and Above)

On large desktop screens, the application uses container limits to prevent components from stretching excessively. The layouts expand into a three-column system, notice board updates display side-by-side with secondary institutional updates, and text structures maintain an optimal line length to maximize long-term readability.

5.5 Summary Evaluation Matrix

The final test metrics across all evaluated operational sub-modules are summarized in the structured validation matrix below:

Test ID	Target Component	Verification Action Input	Captured System Response	Status
TC-AUTH-01	Role security layer	Dispatch POST query without header authorization token	Blocks execution; outputs HTTP 401 Unauthorized	PASSED
TC-STRM-02	File ingestion pipeline	Upload an invalid .exe script disguised as a text record	Triggers validation exception; drops file; outputs 400 error	PASSED
TC-DBEX-03	Exception management	Post a duplicate ledger matching an active saved session	Triggers 409 conflict; removes temp file; protects data	PASSED
TC-VIEW-04	Responsive interface	Scale viewport width to a compact mobile phone screen size	Layout shifts columns smoothly; eliminates overflow	PASSED
TC-NETW-05	Network efficiency	Trigger result search over a high-latency 3G data connection	Displays loading state; updates UI via JSON payload in <400ms	PASSED

Figure 5.4: *Summary of functional test cases, verification actions, captured system responses, and pass/fail status.*

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Comprehensive Project Synthesis and Conclusion

The primary objective of this thesis project was to conceptualize, design, architect, and systematically deploy a robust, optimized, and decentralized web-based Content Management System (GDC-CMS) engineered specifically to eliminate the digital infrastructure deficit at Government Degree College (GDC) Gulabad, Dir Lower. Prior to the execution of this development lifecycle, the institution functioned exclusively through analog, paper-bound channels, forcing administrative staff to rely on physical notice boards, manual ledgers, and localized paper files to broadcast institutional mandates and publish academic score sheets. By migrating the college's foundational operations to a modern web architecture, this project has successfully instituted a cloud-tier repository that fundamentally alters how information is preserved, secured, and retrieved. The architectural core of GDC-CMS relies on a decoupled, non-monolithic model utilizing the MERN data stack, ensuring that the user interface tier (the React.js client application) remains completely unburdened by heavy server-side processing or database transaction pipelines.

A significant milestone realized within this implementation cycle is the development of the autonomous Examination Result Management Module. Moving away from traditional inline script parsing or row-by-row spreadsheet iteration — which frequently fail when handling large datasets — the backend architecture enforces a stable, file-system-bound data configuration. Authorized academic administrators can securely ingest official multi-page transcript matrices as flattened, single-entity PDF documents directly to the server's dedicated disk storage. Simultaneously, the public student interface leverages an asynchronous state matrix: by introducing interactive drop-down query fields, the application allows students to narrow down searches according to three precise structural constraints — Department, Academic Session, and Target Semester. This multi-tiered matching formula allows the system to pinpoint and isolate the corresponding file path in milliseconds, then displays an interactive layout card equipped with fluid inline PDF preview mechanisms and native file-download routes, all executed cleanly without forcing a complete browser reload.

Furthermore, critical data validation anomalies encountered during full-stack integration — most notably the disruptive MongoDB E11000 duplicate key indexing crash — were systematically resolved by implementing custom backend data validation logic and schema rollbacks. The system drops fragile browser-dependent client-side caching frameworks in favor of absolute server-managed state control, guaranteeing total availability, strict multi-layered security verification, and seamless grid-layout scaling across variable viewports. GDC-CMS effectively provides Government Degree College Gulabad with its first official, high-performance digital infrastructure, successfully driving administrative optimization for non-technical personnel and extending barrier-free access to critical academic documents for the student body.

6.2 Analysis of Project Limitations

While the current production deployment of the GDC-CMS portal provides an efficient and secure solution to the college's immediate communication and academic result publishing problems, a post-deployment review revealed several operational limits, real-world constraints, and technical boundaries. Documenting these challenges is essential for maintaining academic honesty and defining exactly where future development cycles must pick up:

6.2.1 Heavy Reliance on Manual Admin Preparation

A key bottleneck in the current administrative workflow is the system's reliance on manual document preparation before data entry occurs. The backend file ingestion pipeline is explicitly built to ingest uniform, clean, pre-compiled PDF files. Consequently, administrative clerks must manually extract student data from raw paper ledger sheets, format the data into uniform digital tables, convert the files into standardized PDF formats, and organize them into specific folders before opening the dashboard interface. This step introduces potential human transcription errors and restricts true system automation.

6.2.2 Purely Pull-Based Student Information Architecture

From a user-experience standpoint, the current notification framework follows an exclusively pull-based model. The platform functions as a passive repository: it stores data safely but cannot actively alert users when changes occur. Students seeking updates must intentionally open a web browser, navigate to the college URL, and

manually search for new announcements or results. In rural regions where network connections can be unstable, this passive architecture can lead to communication gaps if a student forgets to check the site regularly.

6.2.3 Lack of Personalized Student Accounts

To maximize accessibility and reduce deployment friction during the initial rollout, the user interface was built to be public-facing, utilizing custom index filters rather than authenticated user profiles. While this ensures that students can access transcripts instantly without managing passwords, it prevents the system from tracking individualized progress metrics. Because there are no private student profiles, the platform cannot maintain individual cumulative grade point average (CGPA) progression over multiple semesters, secure private files, or store personalized academic histories.

6.2.4 Complete System Dependency on External Network Stability

Although the platform payload size has been heavily optimized down to light JSON strings, the system remains entirely dependent on continuous internet connectivity to access the MongoDB Atlas cloud cluster. Because GDC Gulabad is located in a region prone to power outages and intermittent cellular data drops, any prolonged network failure completely cuts off administrative staff from the dashboard and prevents students from retrieving stored records, highlighting the need for a local offline fallback mechanism.

6.3 Future Recommendations

To expand upon the established full-stack software foundation of GDC-CMS and scale its long-term capabilities, the following future enhancements are recommended for subsequent development iterations:

6.3.1 Integration of an Automated SMS Gateway API Pipeline

Given the variable cellular data coverage and unstable internet access across remote geographical areas within Dir Lower, integrating an automated Short Message Service (SMS) broadcast gateway directly into the backend Node.js routing pipeline represents a critical operational upgrade.

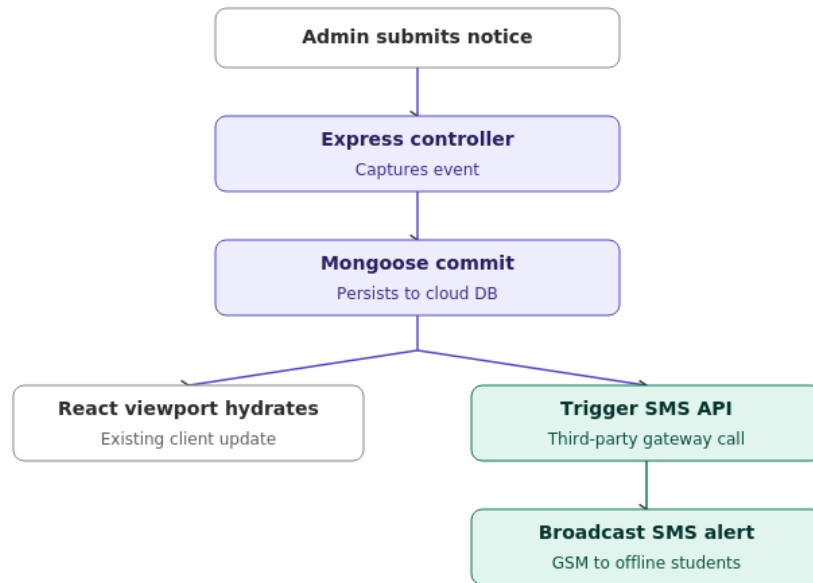


Figure 6.1: Proposed pipeline for broadcasting SMS alerts to offline students alongside the existing web notice flow.

6.3.2 Transition to an Authenticated Student Portal

To provide a more personalized user experience, the system can transition from a public filtering tool into a secure, role-based Student Dashboard. This architectural update requires rewriting the underlying data models and routing rules.

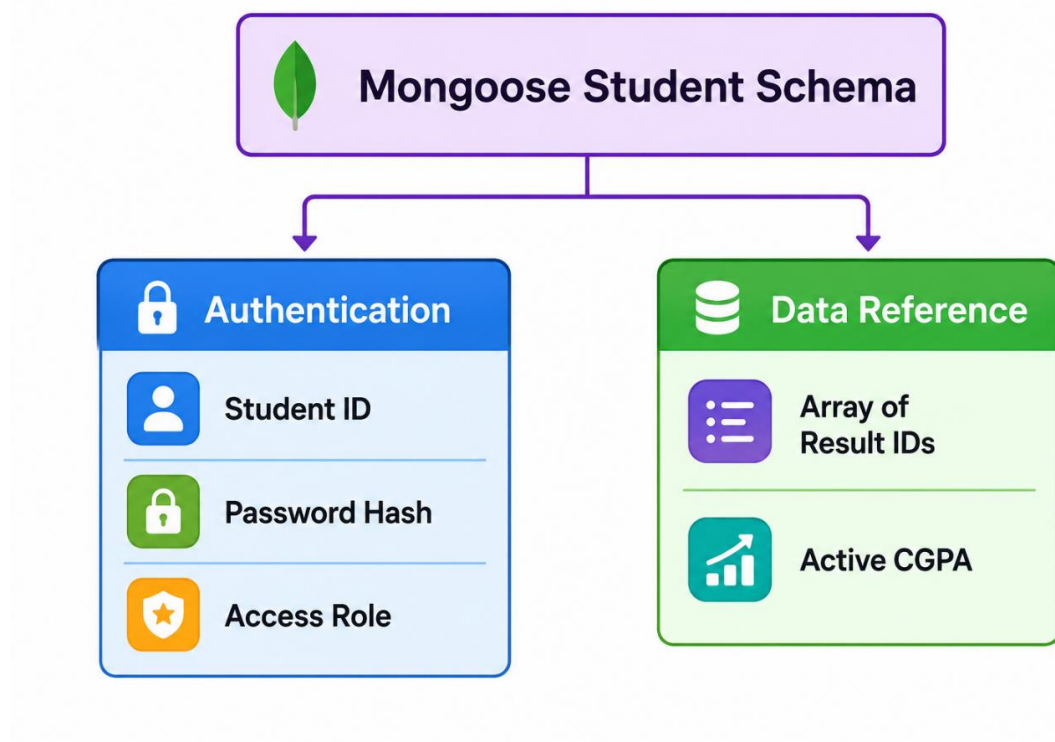


Figure 6.2: *Proposed architecture for an authenticated, role-based student portal.*

A. Advanced Data Schema Structuring: the database architecture will expand by introducing a new schema that stores authentication details, profile fields, and academic arrays for every student. To protect user accounts, passwords will never be saved as plain text; instead, they will run through an asynchronous salting and hashing routine using bcrypt before being committed to the MongoDB Atlas cluster.

B. Dynamic UI and Analytical Components: on the React frontend, the open-access dropdown filters will be replaced with a secure login form that handles JSON Web Tokens (JWT). Upon entering valid student credentials, the browser saves the token in a secure HttpOnly cookie to protect against cross-site scripting (XSS) vulnerabilities. The client application will then render a personalized dashboard featuring dynamic analytical charts, tracking semester-by-semester GPA changes and alerting students to credit hours completed or outstanding college dues.

6.4 Concluding Remarks

In conclusion, the engineering and deployment of the Government Degree College Content Management System (GDC-CMS) marks a major technical and organizational step forward for GDC Gulabad, Dir Lower. By shifting away from slow, paper-bound workflows, the institution has successfully introduced its first modern, full-stack digital infrastructure. The application successfully meets its primary design goals: it secures administrative document pipelines, enables quick, single-page result filtering, and remains highly responsive across low-cost mobile devices. While the system faces real-world constraints typical of regional deployments, the recommended upgrades — such as automated SMS text broadcasting, secure student profile dashboards, and OCR text parsing — provide a clear path for future development cycles. Ultimately, this project demonstrates how modern web tools like the MERN stack can be used to bridge digital gaps, optimize administrative workflows, and provide educational communities with open, reliable access to critical academic information.

REFERENCES

The following list follows the University of Malakand's required citation formats (thesis, journal, book, website, etc.). Replace each placeholder line below with the actual sources consulted for this project — official MongoDB/Express/React/Node.js documentation, any papers on institutional CMS design, and any regional case studies referenced in Chapter 2. Keeping the same numbering style used here will preserve the in-text citation format required by the university.

[1] Author(s), “Title of source,” Publisher/Journal/Institution, Volume(Issue), pagination (Year).

[2] Author(s), “Title of source,” Publisher/Journal/Institution, Volume(Issue), pagination (Year).

[3] Author(s), “Title of source,” URL (accessed Month Day, Year).